VM AVAILABILITY AND DISASTER RECOVERY

A PROJECT REPORT

Submitted by

REGNO	NAME
111624102235	**VARUNESH**
111624102236	**S.VEERAMANIKANDAN**
111624102246	**YOGESH S**
111624102240	**VISHNU VARDHAN S**

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

R. M. K. COLLEGE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution)

PUDUVOYAL



OCTOBER 2026

R.M.K. COLLEGE OF ENGINEERING AND TECHNOLOGY
(An Autonomous Institution)
R.S.M. Nagar, PUDUVOYAL-601 206
Approved by AICTE, New Delhi /Affiliated to Anna University, Chennai
Accredited by NBA, New Delhi (All Eligible Courses)/ NAAC with 'A' Grade
An ISO 21001:2018 Certified Institution

BONAFIDE CERTIFICATE

This is to certify that **VARUNESH, S.VEERAMANIKANDAN, YOGESH S and VISHNU VARDHAN S**, bearing **Register Numbers 111624102235, 111624102236, 111624102246 and 111624102240** respectively, of **3rd Year, 5th Semester, Department of Computer Science and Engineering**, have successfully completed the mini project entitled **"VM Availability and Disaster Recovery"** during the academic year **2026–2027** under my guidance and supervision.

The work submitted in this report is a bonafide record of the work carried out by the student/team as part of the academic requirements of the course.

Name of the Student	**Project Guide**
VARUNESH	Dr. J GNANA JESLIN
S.VEERAMANIKANDAN	
YOGESH S	
VISHNU VARDHAN S	

ABSTRACT

Businesses run their services on virtual machines (VMs) that sit on physical hosts, inside availability zones, inside regions, and every one of these layers can fail. The design of a deployment, how many VMs, in which zones, with what failover and with what copy in another region, decides how much downtime and how much lost data a failure causes, and also decides the cost. Choosing the design usually rests on rules of thumb. This mini project builds a tool that replaces the rules of thumb with calculation and simulation, using only the Python standard library. It provides the closed-form mathematics of availability (MTBF and MTTR, series and parallel systems, a Markov model of an active-passive pair), a Monte Carlo simulator that fails hosts, zones and regions at random and judges six designs on the same failures, four disaster-recovery tiers (backup and restore, pilot light, warm standby, multi-region active-active) with their recovery time (RTO) and data loss (RPO), a backup calculator, and a planner that returns the cheapest design meeting an availability target and RTO and RPO limits. The simulator reproduces the closed-form and Markov-model results within 2.4% in all six comparisons. Over 20,000 simulated years a single VM was down 21.3 hours a year; automatic restart cut this to 4.3 hours; two active VMs in two zones reached 2.5 hours, a floor set by region disasters; and adding a warm-standby DR region reduced downtime to 3.7 minutes a year, with about 1 minute of data lost per disaster. A command-line tool and a browser interface are provided, and 39 automated tests pass.

TABLE OF CONTENTS

S.No	Section	Page No.
1	Introduction	1
2	Objectives	3
3	Tools and Technologies	5
4	Architecture Diagram	6
5	Methodology	8
6	Results	13
7	Conclusion	23
8	Reference	25

1. INTRODUCTION

1.1 Background

Most services that people use every day, from online shops to college portals, run on virtual machines. A virtual machine (VM) is a software computer that runs on a physical host, a host sits in a data centre called an availability zone, and several zones make up a region of a cloud provider. Barham et al. [6] described the hypervisor technique that made this practical, and Clark et al. [7] showed that a running VM can be moved from one host to another with only a very short pause. Every layer of this stack can fail: a host can break, a zone can lose power or cooling, and in rare cases a whole region can become unavailable because of a natural disaster, a large network failure or a faulty software release [4].

The quality that users notice is **availability**, the fraction of time the service works. It is usually stated in "nines": 99.9% allows about 8.8 hours of downtime a year, 99.99% about 53 minutes. Two other numbers matter when something does fail. The **recovery time objective (RTO)** is how long the service may stay down after a disaster. The **recovery point objective (RPO)** is how much recent data the business can afford to lose, measured in time: an RPO of 15 minutes means the newest 15 minutes of data may be lost. Cloud providers publish service level agreements that depend on the way a customer deploys: for example Microsoft documents different availability figures for a single VM, for several VMs in one availability set and for VMs spread over availability zones [10]. The customer is responsible for choosing the design, and the cost grows with every copy of a VM and every copy of the data.

Table 1.1: What each layer of the stack can lose and what protects against it

Failure	Typical duration	How often	Protection
Host (physical server)	hours	every few months per host	automatic restart on another host, a standby VM, or a second active VM
Availability zone	hours	years apart	a second VM in another zone
Region	days	decades apart	a copy of the service and its data in another region (disaster recovery)
Data (deletion, corruption, ransomware)	n/a	unpredictable	backups kept apart from the VM

1.2 Problem Statement

A team that has to choose a deployment faces questions that have numerical answers but are usually answered by habit. How much downtime will a single VM have in a year? How much does a second VM in the same zone help, and how much in another zone? Is automatic restart enough? How much does a slow failover cost? When is a second region worth its price, and which of the standard disaster-recovery patterns meets the RTO and RPO that the business asked for? A spreadsheet can answer the first few questions with formulas, but it cannot handle failover delays, waiting for repairs, or failures that overlap in time, which are exactly the cases that decide the outcome.

The problem addressed here is to build a tool that (i) computes availability from first principles, (ii) simulates random failures of hosts, zones and regions and compares deployment designs on the same failures, (iii) models disaster-recovery tiers with their RTO, RPO and cost, and (iv) recommends the cheapest design that meets stated targets, while (v) being checked against known mathematical results so that its numbers can be trusted.

1.3 Motivation

Three observations motivated the project. First, the rules of thumb contradict each other: redundancy is said to raise availability, yet the residual downtime of a well-built design is dominated by rare, long events that redundancy inside a region cannot touch. A simulation shows which event dominates. Second, availability, RTO, RPO and cost pull in different directions, and a planner that lists every option with the reason it passes or fails makes the trade-off visible. Third, a reliability model is only useful if it can be validated; the simple structure of the problem allows the simulator to be compared with exact formulas and with a Markov model, which most student projects on cloud reliability do not attempt.

1.4 Scope

The tool is a model, not a connection to a real cloud. It treats hosts, zones and regions as independent components that fail and are repaired at random with given mean times. It models six deployment designs and four disaster-recovery tiers, backup schedules and restore times, and the planner described above. It does not call cloud provider APIs, does not model correlated failures other than the zone and region hierarchy, does not model software faults, network partitions or capacity shortages, and treats the recovery times and costs of the DR tiers as input assumptions (Table 5.3) rather than provider prices. The limits and ways to remove them are discussed in Section 7.

2. OBJECTIVES

The main objective is to design and implement a tool that calculates, simulates and plans the availability and disaster recovery of virtual machines, and to evaluate it against known mathematical results. The specific objectives are:

- To implement the mathematics of availability: MTBF and MTTR, nines, series and parallel systems, k-out-of-n systems, and an exact Markov model of an active-passive pair.
- To build a Monte Carlo simulator in which hosts, zones and regions fail and are repaired at random, and in which six deployment designs are evaluated on the same failures.
- To model failover for an active-passive pair and the takeover by a DR region, including the case in which the standby fails during the switch.
- To model four disaster-recovery tiers with their recovery time, data loss and cost, and to report how often a takeover is needed.
- To calculate backup data age and restore time for a given schedule and data size.
- To provide a planner that returns the cheapest design meeting an availability target, an RTO and an RPO, and explains why each rejected design fails.
- To validate the simulator against the closed-form results and to offer a command-line interface and a browser interface.

Table 2.1: Functional requirements

ID	Requirement
FR1	Compute the availability, nines and yearly downtime of a component from its MTBF and MTTR, and of redundant groups
FR2	Simulate host, zone and region failures for a chosen number of years with a fixed random seed
FR3	Evaluate single VM, auto-restart, active-passive (one or two zones), active-active and active-active with a DR region on the same failures
FR4	Report availability, downtime per year, incidents per year and longest incident for every design
FR5	Model DR takeover with the recovery time, data loss and cost of each tier
FR6	Compute the average age of the newest good backup, its 99th percentile and the restore time
FR7	Recommend the cheapest design that satisfies the targets and give the reason for every rejected one
FR8	Reject wrong input (negative repair time, empty targets) with a clear message

Table 2.2: Non-functional requirements

Requirement	How it is met
Correctness	Simulation compared with closed forms and a Markov model; 39 automated tests
Repeatability	Every simulation uses a seed; the same seed gives the same result
Fair comparison	All designs are judged on the same simulated failures (common random numbers)
Portability	Python standard library only
Speed	Twenty thousand simulated years for all six designs in about two seconds

3. TOOLS AND TECHNOLOGIES

The tool is written in Python and uses only the standard library, so that each model can be read next to its formula. Table 3.1 lists what was used.

Table 3.1: Tools and technologies used

Item	Tool	Purpose
Language	Python 3.10 or newer (developed and tested on 3.13)	Mathematics, simulation, planner, interfaces
Libraries	Standard library only: `random`, `heapq`, `dataclasses`, `math`, `http.server`, `json`, `unittest`	Random failures, event timers, server, tests
Browser interface	One HTML file with CSS and plain JavaScript	Four tabs: availability, strategies, planner, backups
Server	`http.server.ThreadingHTTPServer`	JSON endpoints used by the page
Testing	`unittest`	39 automated tests
Charts for this report	matplotlib	Figures 6.1 to 6.6 (not needed to run the tool)
Editor and shell	Visual Studio Code, PowerShell or any terminal	Development

3.1 Running the project

The project is a folder named `vmdr`. No installation is needed. From inside the folder:

```
python -m unittest discover -s tests          # the 39 automated tests (a few
seconds)
python -m vmdr.cli demo                       # every part of the tool with
default values
python -m vmdr.cli avail --mtbf 2000 --mttr 4
python -m vmdr.cli compare --years 5000
python -m vmdr.cli plan --target 0.9999 --rto 60 --rpo 15
python -m vmdr.cli backup --interval 1440 --success 0.98
python -m vmdr.server 8000                    # browser interface at
http://127.0.0.1:8000
python bench.py                               # the experiments of Chapter 6
```

4. ARCHITECTURE DIAGRAM

The tool has four layers (Figure 4.1). The presentation layer is the command line, the browser page and the HTTP server behind the page. The planner holds the DR tiers and the function that chooses the cheapest design. The simulation engine generates random failures, evaluates the six strategies on them and contains the state machine of a failover. At the bottom, the mathematics module holds the closed-form results that the simulation is checked against, and the backup module holds the data-age and restore-time calculations. A layer calls only the layer below it, so the engine can be tested without an interface.

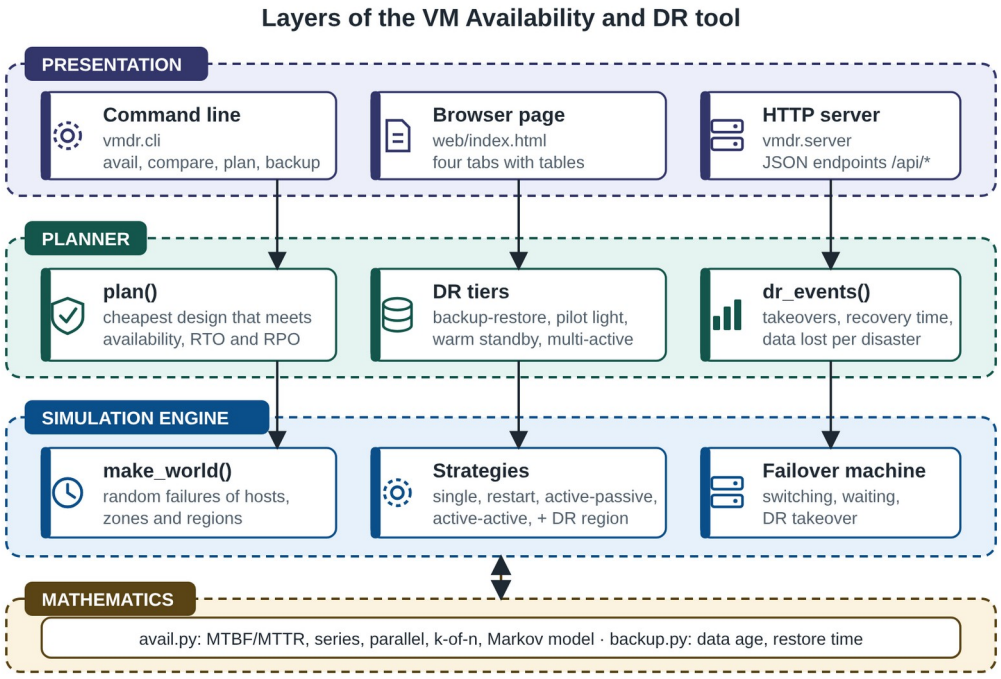


Figure 4.1: Layers of the VM Availability and DR tool

Table 4.1: Module responsibilities

File	Lines	Responsibility
`vmdr/avail.py`	79	Availability formulas and the Markov model of an active-passive pair
`vmdr/sim.py`	217	Failure generator, interval arithmetic, six strategies, failover state machine, DR takeover
`vmdr/dr.py`	93	DR tiers, data-loss sampling, DR event statistics, planner
`vmdr/backup.py`	39	Backup data age, percentile and restore time
`vmdr/cli.py`, `vmdr/server.py`	125	Command line and HTTP server
`web/index.html`	102	Browser page
`tests/test_vmdr.py`	233	39 automated tests
`bench.py`	88	Experiments of Chapter 6

4.1 The deployment that is modelled

Figure 4.2 shows the deployment that the tool models. The primary region has two availability zones, each with a host that can run the VM. A load balancer or failover manager sends users to a working VM. The DR region holds a standby copy of the service whose readiness depends on the tier, and a backup vault holding copies of the disks. Data reaches the DR region by asynchronous replication and by backups. The strategies of Section 5.3 use different parts of this picture: a single VM uses one host in one zone; an active-passive pair uses two hosts; an active-active pair uses one host in each zone; the DR strategy adds the other region.

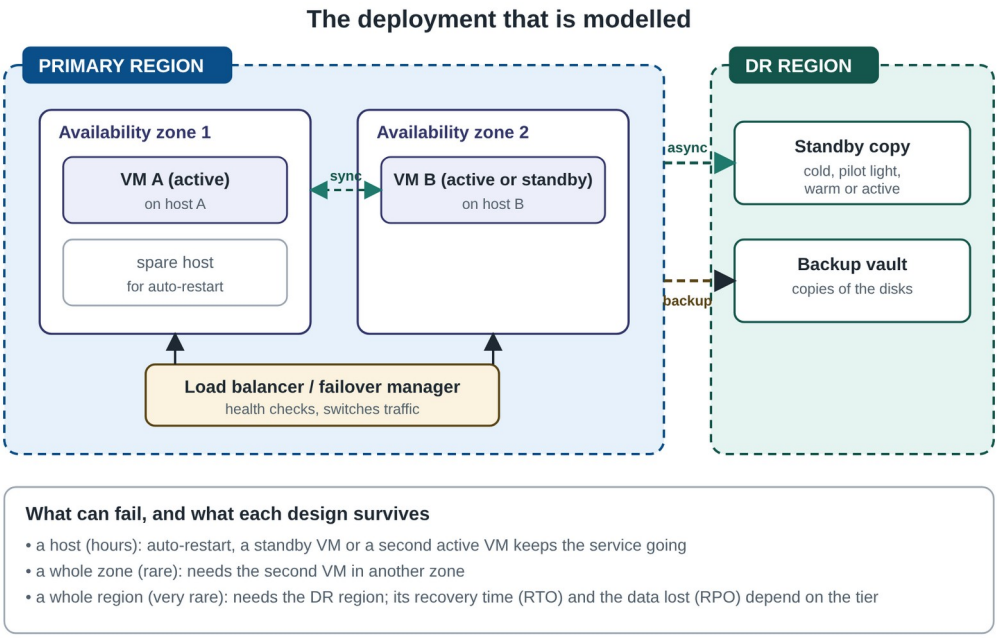


Figure 4.2: The deployment that is modelled

5. METHODOLOGY

5.1 Overall approach

The method combines three ways of looking at the same question. **Closed-form mathematics** gives exact answers for the simple cases, and the **Markov model** gives an exact answer for the active-passive pair. **Monte Carlo simulation** handles everything else, and is validated by reproducing the exact answers where they exist. The **planner** uses the simulation to rank designs against stated targets. This ordering is deliberate: a simulator whose answer is not checked against anything is only a random number generator, so the first results in Chapter 6 are the validation.

5.2 Availability mathematics

A repairable component alternates between working for a mean time MTBF (mean time between failures) and being repaired for a mean time MTTR (mean time to repair). Over a long time its availability is **A = MTBF / (MTBF + MTTR)**; only the two means matter, not the shape of the distributions [1][2]. For a host with MTBF = 2,000 hours and MTTR = 4 hours, A = 0.998004, which is 2.70 nines and about 17.5 hours of downtime a year. Components combine in two ways. In **series** all must work, so availabilities multiply: a VM needs its host and its zone and its region. In **parallel** at least one must work, so unavailabilities multiply: two independent copies each with A = 0.998 give $1 - 0.002^2 = 0.999996$. A group in which at least *k* of *n* identical copies must work has availability the sum over *i* from *k* to *n* of C(n, i) A^i (1 − A)^(n − i).

Parallel redundancy has a limit that the formulas show clearly: it works only for failures that are independent. Two VMs in the same zone share the zone, and two VMs in the same region share the region, so the probability of a zone or region failure appears in series with the redundant group and sets a floor that no number of VMs can lower. The simulator reproduces this effect with a hierarchy of hosts, zones and regions (Section 5.3).

An active-passive pair, in which one VM serves and the other waits, cannot be written with series and parallel formulas because of the failover delay and the fact that the roles change. Under the assumption that failure, repair and failover times are exponential, the pair is a continuous-time Markov chain with four states (Figure 5.1): both up (0), standby down (1), active down and switching (2), both down (3). The service is down in states 2 and 3. The long-run probabilities satisfy the balance equations of the chain, which the program solves by Gaussian elimination, and the unavailability is $\pi_2 + \pi_3$. For a pair with MTBF 2,000 h,

MTTR 4 h and a failover of 2 minutes this gives 2.05×10^{-5}, about 10.8 minutes a year [11].

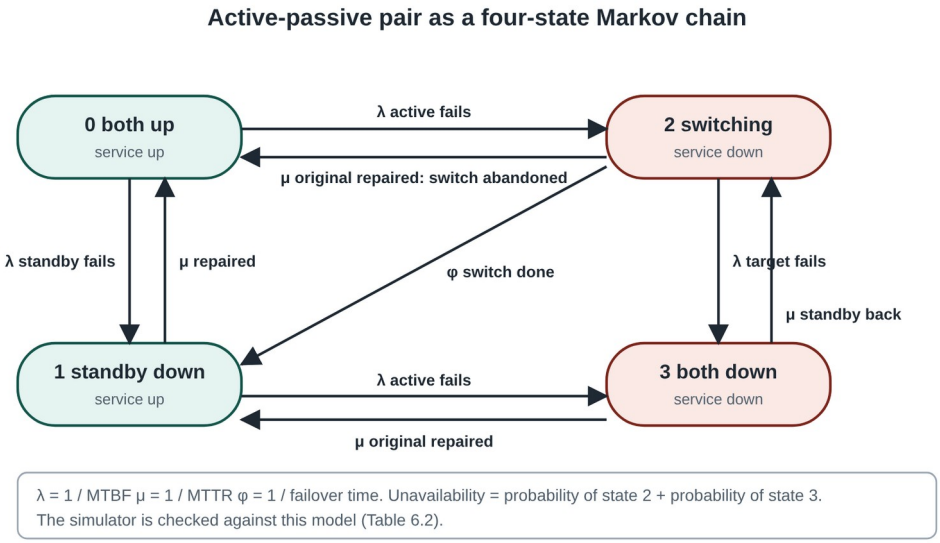


Figure 5.1: Active-passive pair as a four-state Markov chain

5.3 Monte Carlo simulation

A **world** is generated first: for every host, zone and region the simulator draws alternating up and down periods, with exponentially distributed lengths of mean MTBF and MTTR, until the simulated time is used up [3]. The result is a list of down intervals for each component. Because each design is then evaluated on the *same* world, the difference between two designs comes from the design and not from luck (the method of common random numbers). Each design is a rule that turns the component intervals into the intervals in which the service is down, using interval arithmetic: the union of two lists is the time when either is down, the intersection is the time when both are.

Table 5.1: The six designs and how the service-down intervals are built

Design	Service is down when
Single VM	its host, its zone or the region is down (union)
Single VM with automatic restart	like a single VM, but a host failure lasts only the restart time (6 minutes) instead of the repair time; zone and region failures are unaffected. Assumes spare capacity on other hosts.
Active-passive, one zone	the failover state machine of Section 5.4 on two hosts in the same zone, or the region is down
Active-passive, two zones	the same state machine with the two VMs in different zones, or the region is down
Active-active, two zones	both instances are down at the same time (intersection), or the region is down; the load balancer is assumed to react at once
Active-active plus DR region	both instances are down, or a region outage has lasted long enough that the DR takeover is in progress (Section 5.5)

The default parameters of the model are in Table 5.2. They are plausible orders of magnitude for a small cloud deployment, not measurements of any provider, and every one can be changed from the command line or the browser.

Table 5.2: Default parameters of the failure model

Parameter	Value	Meaning
Host MTBF, MTTR	2,000 h, 4 h	a host fails about every 83 days and takes four hours to repair
Zone MTBF, MTTR	17,520 h, 3 h	a zone outage about every two years, lasting three hours
Region MTBF, MTTR	175,200 h, 48 h	a regional disaster about every twenty years, lasting two days
Restart time	6 min	time for HA to detect a dead host and boot the VM elsewhere
Failover time	2 min	time to switch from the active VM to the standby
Detection time	5 min	time to notice a region outage and decide to fail over

5.4 The failover state machine

For an active-passive pair the service-down intervals are not a simple combination of the two VMs' intervals, because the standby takes over only after a delay and then the roles stay swapped. The simulator therefore processes the failure and repair events of the two VMs in time order together with failover timers. When the active VM goes down and the other is up, the service is down and a timer of the failover time starts; when it ends, the other VM becomes active. If the target fails before the timer ends, the switch is cancelled. If the original VM comes back first, the switch is abandoned and the service resumes at once. If both are down, the service waits for the first repair and then starts a switch if it was the standby that returned. The old active VM, once repaired, simply becomes the standby (there is no fail-back). Six hand-checked cases in the tests cover each branch.

5.5 Disaster-recovery tiers

A region outage is survived by moving the service to the DR region. For each outage of the primary region the simulator checks whether it lasts longer than the takeover time T = detection time + the tier's restore time. If it does not, nothing is moved and the service is down for the whole outage. If it does, the service is down for T and then runs in the DR region until the primary returns; if the DR region is itself down at the moment of takeover, the service waits for the first recovery, and if it fails while carrying the service the downtime is counted. At every takeover some recent data is lost, because it had not yet reached the DR region. For the asynchronous tiers the amount is exponentially distributed

with the replication lag as its mean; for backup and restore it is uniform over the backup interval; asynchronous replication of VMs is described in [8] and the tier patterns in [9]. The four tiers follow the patterns described by cloud providers [9].

Table 5.3: Disaster-recovery tiers (model assumptions)

Tier	Restore time	Data lost per takeover (mean)	Extra cost	Description
Backup and restore	240 min	720 min	0.1	daily backup copied to another region; the VM is rebuilt from the backup after a disaster
Pilot light	60 min	5 min	0.3	data is replicated continuously, but the VMs in the DR region are switched off and started on demand
Warm standby	15 min	1 min	1.0	a smaller copy of the service runs all the time in the DR region and is scaled up on demand
Multi-region active-active	2 min	0.1 min	2.1	both regions serve users; the surviving region takes all the traffic

The restore times, the replication lags and the costs are assumptions of the model, stated in the style of provider guidance. They are inputs, not findings, and should be replaced with a provider's own figures when a real design is evaluated. Cost is measured in VM equivalents, with one VM costing 1.0; the primary site of two VMs and a load balancer costs 2.1, and the tier adds its extra cost on top.

5.6 Backups

A backup taken every I minutes that always succeeds leaves, at a random moment, a newest copy that is on average $I/2$ old and at worst I old. If each backup succeeds only with probability p, the age adds one interval for every failed backup in a row, so the mean age is $I \times (1/p - 1/2)$ and the age that is exceeded one time in 100 is $(k + 1) \times I$, where k is the smallest number of failures in a row whose probability is at most 1%. The restore time is the time to copy the data (size ÷ throughput) plus the boot and checking time, taken as 20 and 10 minutes. A Monte Carlo check of the mean is included in the tests.

5.7 The planner

The planner evaluates all designs on one simulated world: the five designs without a DR region and the DR strategy with each of the four tiers. For each it records availability, cost, the recovery time for a region disaster and the data loss. A design without a DR region can survive a region disaster only by waiting for the region to be repaired, so its recovery time is the region's repair time and it has no copy of the data outside the region; it therefore fails any RTO or RPO shorter than that. A design is accepted only if it meets the availability target and every limit that was given; the cheapest accepted design is recommended, and

every rejected design carries the list of limits it breaks. If nothing is accepted the planner says so rather than choosing the nearest.

5.8 Testing methodology

Five kinds of test were used. **Mathematics**: known values (a 99.99% target allows 52.56 minutes a year, 2-of-3 with A = 0.9 is 0.972) and the limits of the Markov model (never better than ideal active-active, worse as failover slows). **State machine**: six hand-calculated cases for failover, including the standby failing after the switch, the target failing during the switch, and an outage still open at the end of the simulation. **DR**: takeover timing, the short outage that causes no takeover, the DR region being down, and the sample means of data loss. **Validation against theory**: the simulated unavailability of a single VM, of an active-active pair and of an active-passive pair must be within 4%, 5% and 8% of the exact value on 15,000 to 30,000 simulated years. **Planner and interfaces**: the recommendations for four known cases, a reason for every rejection, and the HTTP endpoints, including the 400 reply for invalid input.

Table 5.4: Cost of the calculations (Y = simulated years, e = failure events per year)

Calculation	Time	Remark
Generate the world	O(Y · e)	one random draw per event
Single, restart, active-active	O(Y · e)	interval union and intersection of sorted lists
Active-passive	O(Y · e log(Y · e))	sorting of events, a heap of failover timers
DR takeover	O(outages · log DR outages)	regional events are very rare
Markov model	constant	a 4 × 4 linear system

6. RESULTS

6.1 Test results

All 39 automated tests pass (`python -m unittest discover -s tests`, run time a few seconds because several tests simulate tens of thousands of years). Table 6.1 lists them by group.

Table 6.1: Automated tests

Group	Tests	What is checked
Availability mathematics	6	A = MTBF/(MTBF+MTTR); nines; series, parallel and k-of-n; invalid input; limits of the Markov model; allowed downtime of an SLA
Interval arithmetic	2	union, intersection, total; failure generator is ordered and inside the horizon
Failover state machine	6	simple failover; standby fails after the switch; target fails during the switch; both down; original back before the switch ends; outage open at the horizon
Disaster recovery	6	takeover time; short outage; DR region down at takeover; DR region fails while carrying the service; data-loss means; tiers ordered by speed and cost
Simulation against theory	8	single VM and active-active against closed forms; active-passive against the Markov model; restart effect; more redundancy never hurts; region floor; same seed same result; unknown strategy
Backups	3	formula against Monte Carlo; percentile; restore time
Planner	6	modest target is met cheaply; four nines needs a DR region; RTO limit selects warm standby; tight RPO selects multi-region; impossible target gives no recommendation; a reason for every rejection
Interfaces	2	command line; HTTP endpoints and the 400 reply

6.2 The simulation reproduces the mathematics

Table 6.2 compares the simulated unavailability with the exact value for six cases: a single VM and an active-active pair (closed forms, 20,000 simulated years) and four active-passive pairs (Markov model, 60,000 years, with zone and region failures switched off so that only host failures remain). The largest difference is 2.4%, which is within the sampling error of a simulation of this length, since the downtime of a design that fails rarely is itself a random quantity. The exact models and the simulator are written independently and agree, which is the main evidence that both are right.

Table 6.2: Simulated unavailability against exact results

Case	Reference	Exact	Simulated	Difference
Single VM, one zone	closed form	2.440e-03	2.432e-03	-0.35%
Active-active pair in two zones	closed form	2.786e-04	2.725e-04	-2.19%
Active-passive pair, MTBF 2000 h, MTTR 4 h, failover 2 min	Markov chain	2.048e-05	2.080e-05	+1.57%
Active-passive pair, MTBF 2000 h, MTTR 4 h, failover 10 min	Markov chain	8.382e-05	8.581e-05	+2.37%
Active-passive pair, MTBF 500 h, MTTR 8 h, failover 2 min	Markov chain	3.133e-04	3.101e-04	-1.03%
Active-passive pair, MTBF 1000 h, MTTR 24 h, failover 5 min	Markov chain	6.304e-04	6.348e-04	+0.69%

6.3 Six designs on the same failures

Table 6.3 and Figure 6.1 give the result of 20,000 simulated years with the default parameters of Table 5.2 and the warm-standby DR tier. The vertical bars in the figure show the range over five different random seeds, which is narrow, so the ranking does not depend on luck.

Table 6.3: Six designs on the same simulated failures

Design	Availability	Downtime (min/year)	Incidents per year	Mean incident (min)	Longest
Single VM, one zone	99.7565%	1280.1	4.89	262	400 h
Single VM with automatic restart on another host	99.9509%	257.9	4.89	53	400 h
Active-passive pair in one zone	99.9538%	243.0	4.91	50	400 h
Active-passive pair in two zones	99.9701%	157.1	4.91	32	400 h
Active-active pair in two zones	99.9719%	147.4	0.07	1994	400 h
Active-active in two zones plus a DR region	99.9993%	3.7	0.07	49	12 h

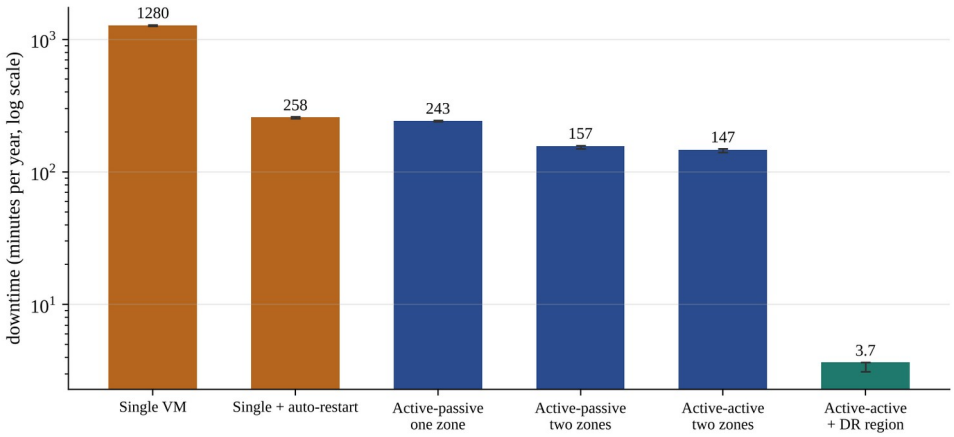


Figure 6.1: Downtime per year of the six designs (log scale; bars show the range over five seeds)

A single VM is down about 21.3 hours a year. **Automatic restart** removes most of the host-repair downtime and brings it to 258 minutes, 80% less, for almost no extra cost. A

second VM adds little over restart in the same zone (243 minutes) because the zone and the region are still shared; putting the second VM in another zone gives 157 minutes, and active-active across two zones, which needs no failover, 147 minutes. At that point something changes. The number of incidents per year drops from about 4.9 to 0.07, but the average incident lasts over 30 hours: the remaining downtime is the rare regional disaster, which no arrangement of VMs inside one region can survive. A DR region removes it: downtime falls to 3.7 minutes a year (99.9993%), and the longest incident from 400 hours to 12.0 hours. This is the main result of the project: past the second nine, availability is bought by protecting against the rare large failure and not by adding more VMs.

6.4 Failover time

Figure 6.2 shows how the downtime of an active-passive pair grows with the failover time, for host failures only (the setting of the Markov model). The downtime is almost proportional to the failover time: 6.3 minutes a year at one minute, 23.6 at five and 234 at an hour, because each of the roughly 4.4 host failures a year costs one failover. The simulation and the Markov model agree up to a failover time of about ten minutes; at 60 minutes the simulation is 11% above the model. The difference is expected and is not an error: the model assumes that the failover time is exponentially distributed, with a few very long switches, while the simulation uses the same fixed time every time, and the two behave differently when the failover is long compared with the repair. For a pair in two zones with all failure types (the downtime column of the sweep) the downtime rises from 152 to 403 minutes a year between a one-minute and a one-hour failover, so a failover of an hour is worse than automatic restart (258 minutes).

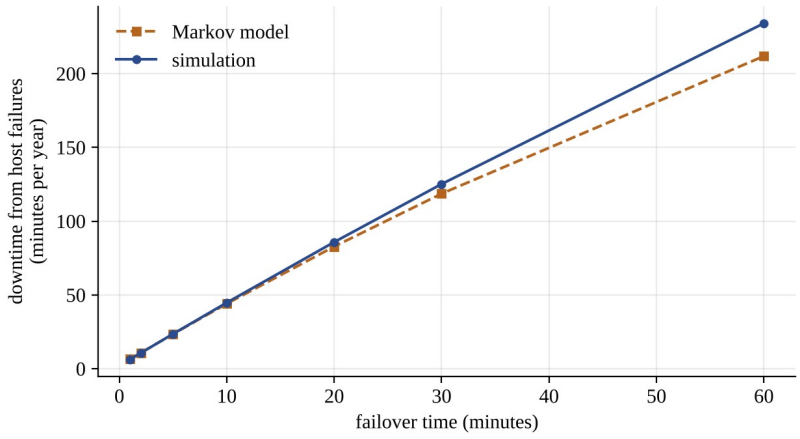


Figure 6.2: Downtime from host failures against the failover time (active-passive pair)

6.5 Repair time

Figure 6.3 varies the host repair time. For a single VM the downtime grows in proportion to the repair time, from 502 minutes a year at one hour to 12,527 at 48 hours. Automatic restart does not depend on it (265 to 255, because the VM is restarted elsewhere and never waits for the repair). Active-passive and active-active pairs are nearly flat while repairs are short, but rise when repairs are slow (163 minutes at 12 hours, 451 at 48 hours) because the second host is more likely to fail while the first is still being repaired. At a repair time of 48 hours restart on spare capacity (255 minutes) beats a pair of VMs that wait for their hosts to be fixed (451 minutes). The lesson is that the speed of recovery of a host matters as much as redundancy. The restart result depends on the assumption that spare capacity exists on another host.

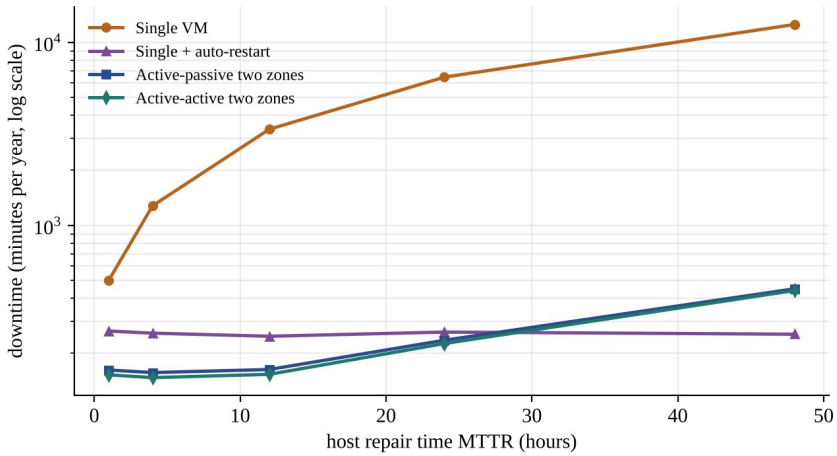


Figure 6.3: Downtime against the host repair time (log scale)

6.6 Disaster-recovery tiers

Table 6.4 and Figure 6.4 show the four tiers. A region disaster occurred 0.051 times a year in the simulation, about once in twenty years, as set by the parameters. The tiers differ by orders of magnitude in recovery time and data loss, and by a factor of 20 in extra cost. The mean data loss measured in the simulation agrees with the model value of Table 5.3 for every tier (for backup and restore 730 against 720 minutes). Backup and restore recovers in about four hours and loses half a day of data, which is acceptable only for systems that tolerate both; pilot light reaches an RTO of about an hour at a small extra cost; warm standby brings it to 20 minutes and about a minute of data loss; multi-region active-active reaches seven minutes (mostly the detection time) and a tenth of a minute of data loss at more than twice the cost. The downtime per year shown in the last column is dominated by

the disaster takeover itself: 14.5 minutes for backup and restore against 3.0 for multi-region.

Table 6.4: Disaster-recovery tiers on 20,000 simulated years

Tier	RTO (min)	Mean data lost (min)	Extra cost	Takeovers per year	Downtime (min/year)
Backup and restore	245	730.45	0.1	0.046	14.5
Pilot light	65	5.04	0.3	0.050	5.9
Warm standby	20	1.01	1.0	0.051	3.7
Multi-region active-active	7	0.10	2.1	0.051	3.0

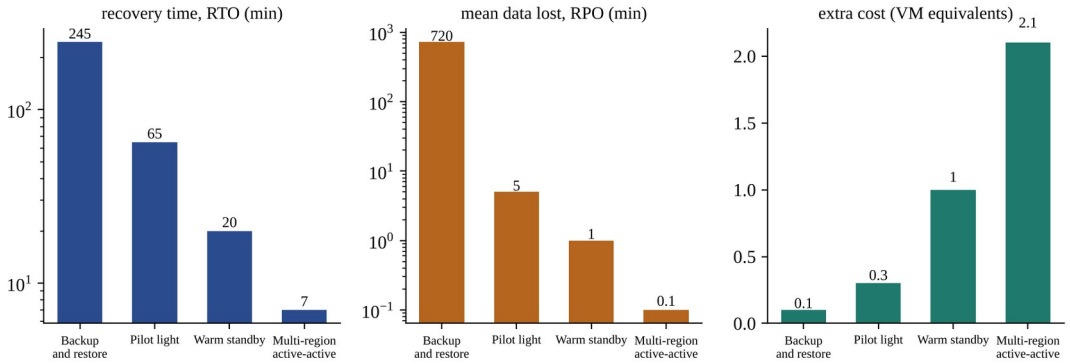


Figure 6.4: Recovery time, data loss and cost of the four tiers (the first two on a log scale)

6.7 Backups

Table 6.5 gives the average age of the newest good backup for different schedules, together with the Monte Carlo check of the formula (100,000 trials each, always within 0.4% of the formula). An unreliable backup job is a hidden risk: a daily backup that succeeds 98% of the time has a mean data age of 749 minutes and a 1-in-100 worst case of three days, while the same schedule with a perfect job gives 720 minutes and one day. Moving from daily to hourly backups divides the data loss by 24 (Figure 6.5). The restore time depends on the data size: at 100 MB/s it takes about 47 minutes for 100 GB, 115 minutes for 500 GB, 3.3 hours for 1 TB, 14.7 hours for 5 TB, so a 4-hour RTO for backup and restore is realistic only up to roughly a terabyte of data.

Table 6.5: Age of the newest good backup

Interval	Success rate	Mean age (formula, min)	Mean age (simulated, min)	99th percentile (min)
60 min	1.00	30.0	30.0	60
240 min	1.00	120.0	120.1	240
1440 min	1.00	720.0	720.3	1440
60 min	0.98	31.2	31.3	180
240 min	0.98	124.9	125.3	720
1440 min	0.98	749.4	751.7	4320
60 min	0.90	36.7	36.8	180
240 min	0.90	146.7	147.1	720
1440 min	0.90	880.0	882.3	4320

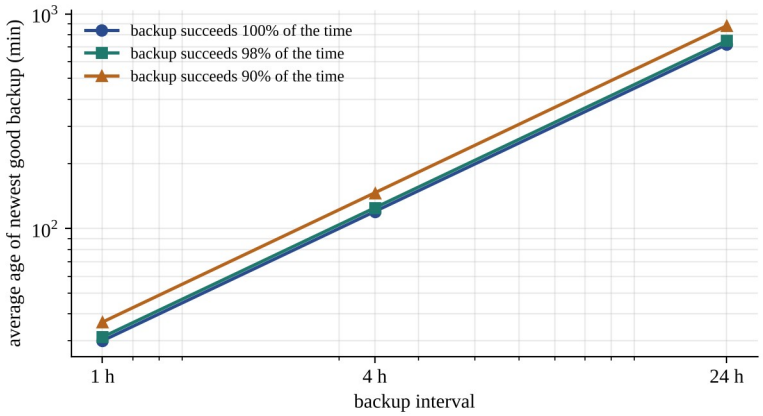


Figure 6.5: Average data age against the backup interval

6.8 The planner

Table 6.6 shows the planner on four typical requirements. An internal tool that needs 99.9% and has no recovery limits is served by automatic restart at a cost of 1.1 VMs. A web shop that needs four nines, an RTO of an hour and an RPO of 15 minutes needs a DR region, and the cheapest tier whose recovery time is within an hour is warm standby: pilot light fails because its recovery time of 65 minutes is five minutes above the limit, and backup and restore fails both limits. A payment service with an RTO of 15 minutes and an RPO of one minute can only be served by multi-region active-active at 4.2 VMs, and it is also the choice when the target is raised to five nines with the same limits. Figure 6.6 shows the options of the web-shop case: cost rises with every step, and only the DR designs pass the four-nines line.

Table 6.6: Planner results

Case	Availability target	RTO limit	RPO limit	Recommended design	Cost
internal tool	99.900%	-	-	restart	1.1
web shop	99.990%	60 min	15 min	active-active + DR: warm-standby	3.1
payments	99.990%	15 min	1 min	active-active + DR: multi-active	4.2
five nines	99.999%	15 min	1 min	active-active + DR: multi-active	4.2

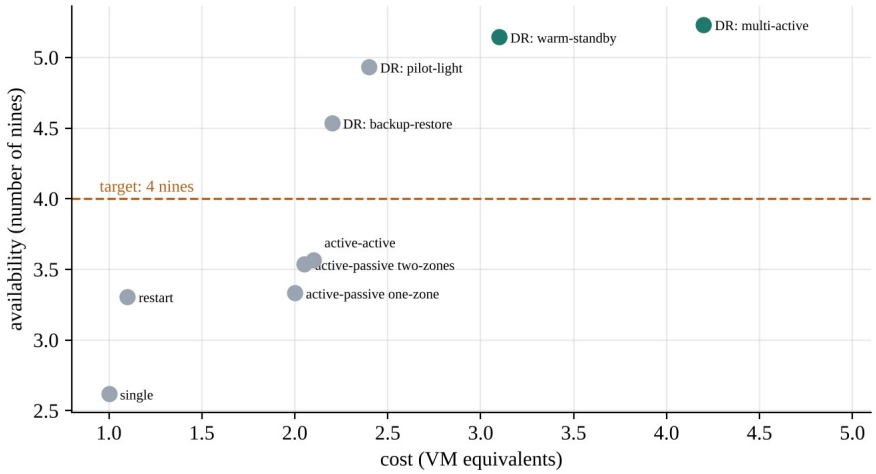


Figure 6.6: Cost against availability for the web-shop case (green: meets all limits)

6.9 Browser interface

Figures 6.7 to 6.10 show the four tabs of the browser interface (`python -m vmdr.server`).

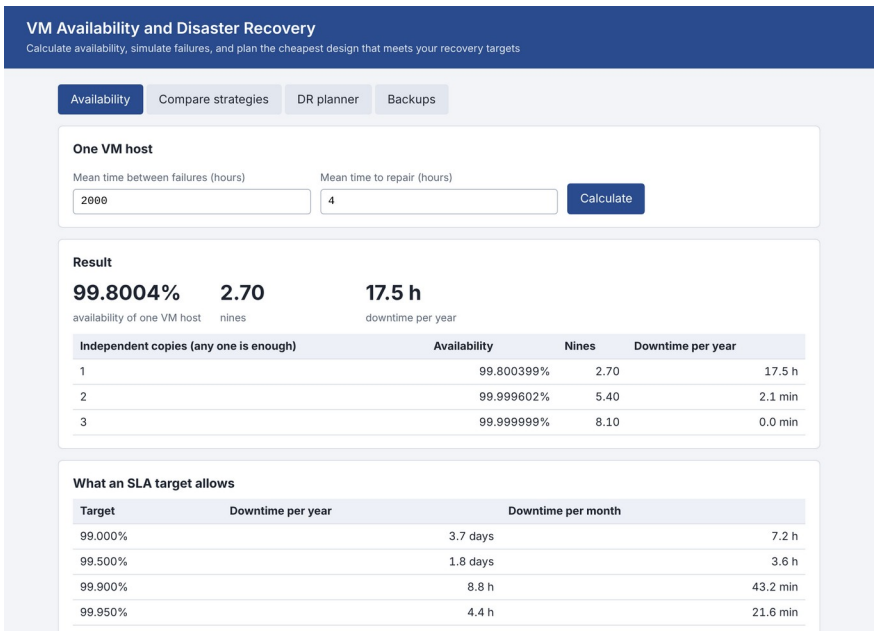


Figure 6.7: Availability tab: one host, redundant copies and the downtime allowed by common targets

In Figure 6.7 a host with MTBF 2,000 h and MTTR 4 h has 99.8004% availability, 2.70 nines and 17.5 hours of downtime a year. Two independent copies reach 99.9996% (5.4 nines) and three about 8 nines; the second table lists what the usual targets allow, for example 52.6 minutes a year for 99.99%.

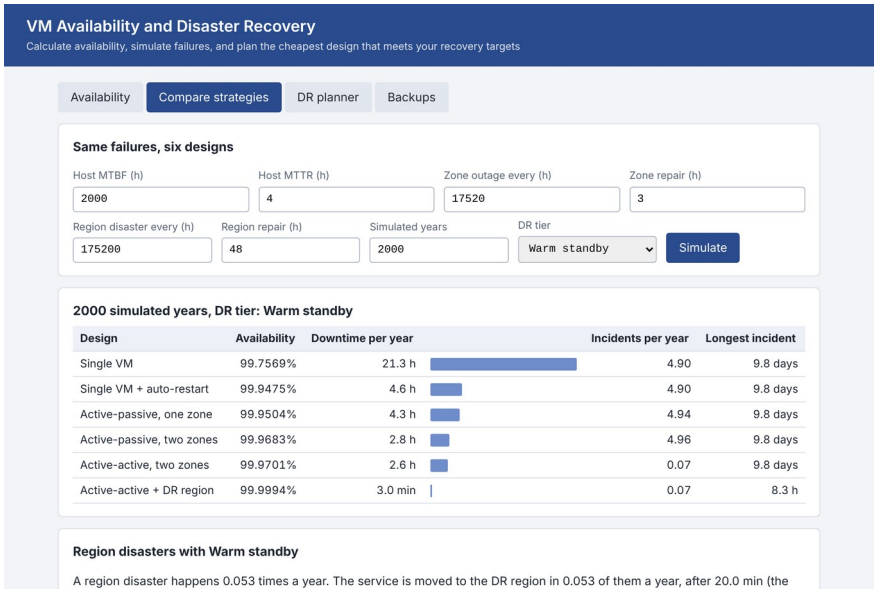


Figure 6.8: Compare tab: six designs simulated over 2,000 years with the warm-standby tier

Figure 6.8 runs the six designs for 2,000 years with editable failure parameters and a choice of DR tier. The bars show the downtime, and the text under the table reports how often a region disaster occurs, the recovery time and the mean data lost.

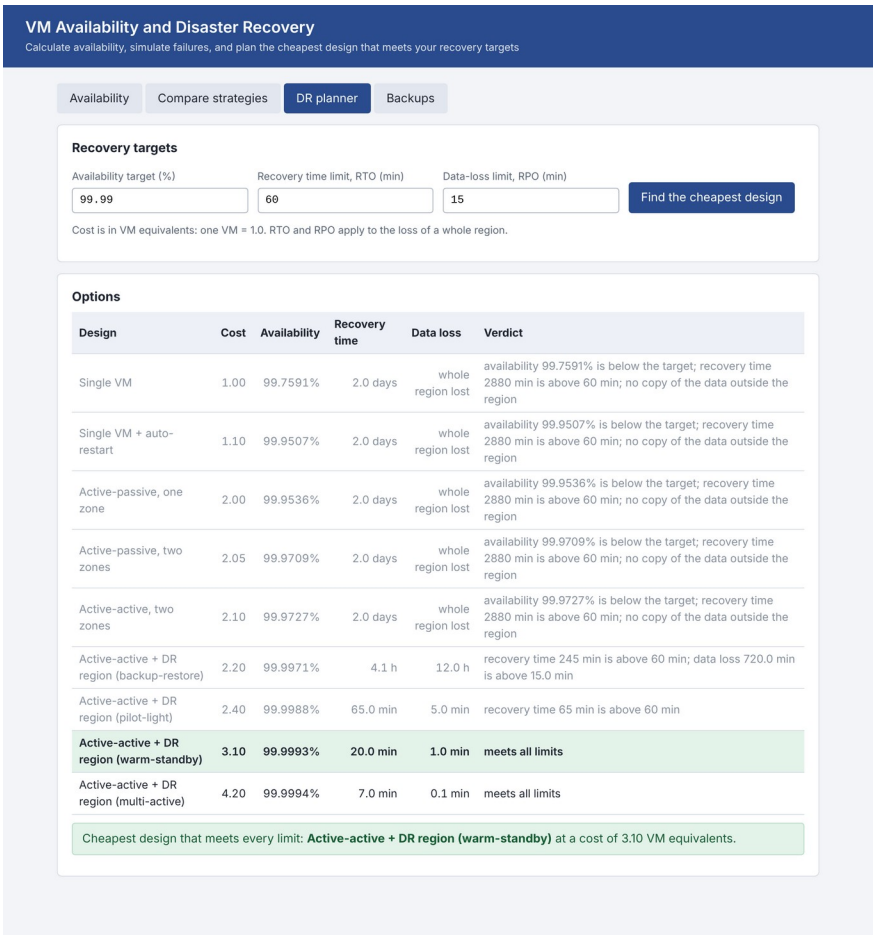


Figure 6.9: Planner tab: targets of 99.99%, RTO 60 minutes and RPO 15 minutes

In Figure 6.9 the planner lists every option with its cost, availability, recovery time and data loss. The grey rows fail, each with its reasons; the two rows in normal colour meet all limits, and the highlighted row, warm standby at a cost of 3.10, is the cheapest of them.

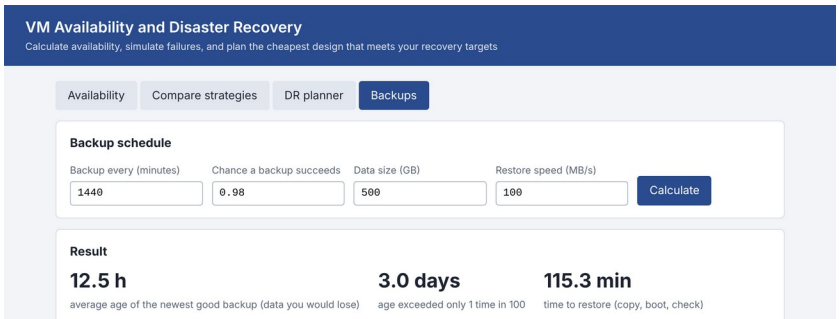


Figure 6.10: Backups tab: data age and restore time

Figure 6.10 shows a daily backup that succeeds 98% of the time: the average age of the newest good backup is 12.5 hours, one time in 100 it is 3.0 days old, and restoring 500 GB at 100 MB/s takes 115 minutes.

6.10 Performance

Twenty thousand simulated years for all six designs take 2.0 seconds, and the planner, which evaluates nine options on 4,000 years, answers in 0.4 seconds, which is quick enough for the browser interface. The times were measured on one machine and one Python version and show the order of magnitude only.

6.11 Discussion

The results support the aims of the project. The simulator agrees with exact mathematics in every case where an exact answer exists, so the numbers for the cases without one can be trusted to the same degree. Three findings stand out. First, the downtime of a design is shaped by its weakest rare event: below roughly 99.97% availability the figure is set by region disasters, and only a DR region lowers it. Second, the quality of recovery matters as much as redundancy: a slow failover or a slow repair can make a pair of VMs worse than a single VM that restarts quickly on spare capacity. Third, the DR tier is a trade-off with no free option: each step cuts the recovery time and data loss by a large factor and raises the cost, and the planner's job is to find the cheapest step that still meets the limits.

The limits of the evidence should be stated just as clearly. The failure rates, the repair times and the tier parameters are assumptions of the model and not measurements of a cloud provider, so the absolute figures (21 hours, 3.7 minutes) would change with real data, and only the relationships between the designs carry over. The model treats failures as independent apart from the zone and region hierarchy, and uses exponential times; real outages are often correlated and heavy-tailed, and software and operator faults, which Gray [5] found to be a leading cause of failures, hit all copies at once, so redundancy would help less than here.

7. CONCLUSION

This mini project built a tool for the availability and disaster recovery of virtual machines. It combines the closed-form mathematics of availability and an exact Markov model of an active-passive pair with a Monte Carlo simulator that fails hosts, zones and regions at random, judges six deployment designs on the same failures, models four disaster-recovery tiers and calculates backups and restores. A planner uses the models to recommend the cheapest design that meets an availability target and RTO and RPO limits, and to explain why the others do not. The simulator reproduces the exact results within 2.4% in six comparisons, and 39 automated tests pass.

The experiments give three practical messages. A single VM was down 21 hours a year; automatic restart on spare capacity removes about 80% of that at almost no extra cost; and beyond the second VM the downtime that remains is the rare regional disaster (2.5 hours a year), which only a DR region can remove (3.7 minutes with warm standby). The speed of failover and repair matters as much as the number of copies. And the DR tier is chosen by the RTO and RPO that the business needs: warm standby for an hour and 15 minutes, multi-region for minutes and seconds, at a cost that grows with each step.

7.1 Limitations

- The model uses assumed failure rates, repair times, tier parameters and costs; it is not calibrated on a real cloud provider.
- Failures are independent except through the zone and region hierarchy, and times are exponential; correlated and heavy-tailed outages are not modelled.
- Software faults, network partitions, capacity shortages and human error, which cause many real outages, are outside the model.
- Auto-restart assumes spare capacity on another host; the load balancer is assumed to react at once.
- A region disaster is modelled as an outage; data destruction and ransomware are treated only through backups.
- The costs are relative VM equivalents, not prices.

7.2 Future scope

- Calibrate the model with real incident data and published provider figures, and add confidence intervals to the simulated results.

- Model correlated failures (common power, common software release) and heavy-tailed repair times.
- Add capacity planning for auto-restart, so that the spare capacity it needs is part of the cost.
- Add more topologies: three zones, k-out-of-n clusters behind a load balancer, and multi-region active-active with traffic splitting.
- Connect the planner to a cloud provider's price list and to the configuration of a real deployment, so that recommendations come with a monthly cost.
- Add a failover drill simulator that estimates the RTO of a procedure from the time taken by each step.
- Include the time to detect failures, and the effect of false alarms, in the failover model.

8. REFERENCE

[1] K. S. Trivedi, Probability and Statistics with Reliability, Queuing, and Computer Science Applications, 2nd ed. New York, NY: Wiley, 2002.

[2] A. Birolini, Reliability Engineering: Theory and Practice, 8th ed. Berlin: Springer, 2017.

[3] A. M. Law, Simulation Modeling and Analysis, 5th ed. New York, NY: McGraw-Hill, 2015.

[4] L. A. Barroso, J. Clidaras and U. Hölzle, The Datacenter as a Computer: Designing Warehouse-Scale Machines, 3rd ed. San Rafael, CA: Morgan & Claypool, 2018.

[5] J. Gray, "Why do computers stop and what can be done about it?" Tandem Computers, Technical Report 85.7, 1985.

[6] P. Barham et al., "Xen and the art of virtualization," in Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP), 2003, pp. 164–177.

[7] C. Clark et al., "Live migration of virtual machines," in Proceedings of the 2nd USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2005, pp. 273–286.

[8] B. Cully et al., "Remus: High availability via asynchronous virtual machine replication," in Proceedings of the 5th USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2008, pp. 161–174.

[9] Amazon Web Services, "Disaster Recovery of Workloads on AWS: Recovery in the Cloud," AWS Whitepaper (backup and restore, pilot light, warm standby and multi-site active-active). [Online]. Available: https://docs.aws.amazon.com/

[10] Microsoft, "Service Level Agreements (SLA) for Online Services" and "Azure Well-Architected Framework: Reliability," Microsoft documentation. [Online]. Available: https://learn.microsoft.com/azure/

[11] G. Bolch, S. Greiner, H. de Meer and K. S. Trivedi, Queueing Networks and Markov Chains: Modeling and Performance Evaluation with Computer Science Applications, 2nd ed. Hoboken, NJ: Wiley, 2006.